# 1. Всё приложение одним взглядом

RateRadar KZ показывает 12 обменников с демонстрационными курсами USD. Выбираешь город, открываешь обменник, добавляешь его в избранное. Данные записаны в коде; интернет и база сейчас не используются.

```
Android запускает MainActivity
  -> RateRadarTheme задаёт оформление
     -> RateRadarNavGraph выбирает экран
        -> List: фильтр городов + карточки
        -> Details: сведения об одном обменнике
        -> Favorites: только избранные карточки
```

Android - система. Kotlin - язык. Compose - библиотека, которая строит интерфейс по Kotlin-функциям. Material 3 - готовые кнопки, панели, карточки и тема. Android Studio - редактор и среда запуска. Gradle - инструмент сборки приложения.

## Где лежит что

| Файл / папка | Его работа |
|---|---|
| MainActivity.kt | Одна Activity, внутри которой живут все три экрана. |
| model/ExchangeOffice.kt | Описание одного обменника: id, имя, город, адрес, часы, два курса, время. |
| data/SampleData.kt | Список обменников и городов. |
| navigation/ | Маршруты, переходы, общее избранное. |
| ui/screens/ | Три функции экранов. |
| ui/components/ | OfficeCard, RateBlock, EmptyState. |
| ui/theme/ | Color: цвета; Theme: выбор схемы; Dimens: размеры; Type: стили текста. |
| res/ | Строки, рисунки, иконка и XML-тема окна. |

Все Kotlin-файлы приложения находятся в app/src/main/java/kz/edu/kbtu/rateradar/. Имя папки java не запрещает Kotlin. package задаёт адрес объявлений, import позволяет писать короткие имена библиотечных функций. Импорты не запускают код.

> **Читай по порядку с открытым проектом. После страницы объясни её без PDF. Затем сделай четыре изменения со страницы 9. Аудит требований - страница 10.**

# 2. Kotlin: синтаксис, который здесь нужен

```
val id: Int = 3
var city: String = "All"
city = "Astana"
fun details(officeId: Int) = "details/$officeId"
```

val - нельзя присвоить другое значение; var - можно. После : указан тип. Int - целое, Double - дробное, String - текст, Boolean - true/false. Тип часто выводится автоматически. = присваивает, == сравнивает. fun объявляет функцию. details(3) вызывает её и получает строку «details/3»: $ вставляет значение.

| Запись | Как читать |
|---|---|
| office.name | Свойство name объекта office. |
| { ... } | Тело функции либо лямбда - передаваемая функция. |
| ( ... ) | Параметры объявления или аргументы вызова. |
| (Int) -> Unit | Функция принимает Int и не возвращает полезный результат. |
| () -> Unit | Такая же функция, но без входных параметров. |
| { id -> ... } | Параметр лямбды называется id; дальше её действия. |
| { it.city == city } | it - единственный параметр лямбды с опущенным именем. |
| Set<Int> / List<...> | Множество уникальных id / упорядоченный список объектов. |
| id in favoriteIds | Проверить наличие id в наборе. |
| Bundle? / ?. / ?: | Значение может быть null / безопасный доступ / запасное значение. |

## Передать действие и выполнить действие - разные вещи

```
IconButton(onClick = onOpenFavorites) { Icon(...) }
OfficeCard(onClick = { onOfficeClick(office.id) }, ...)
```

onClick получает функцию, чтобы выполнить её при нажатии. Запись onOpenFavorites() вызвала бы её сразу при построении UI. Последние фигурные скобки у IconButton описывают содержимое кнопки. Это другой параметр, не реакция на нажатие.

Именованный аргумент text = office.name объясняет, что именно передаётся. В объявлении modifier: Modifier = Modifier знак = задаёт значение по умолчанию. Запятая разделяет аргументы; последняя запятая разрешена. Скобки просто открывают и закрывают соответствующие блоки.

В примерах с ... оставлены только нужные части вызова. Многоточие не является исполняемым Kotlin-кодом.

# 3. Android, Compose и Scaffold

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            RateRadarTheme { RateRadarNavGraph() }
        }
    }
}
```

MainActivity наследуется от ComponentActivity. Android вызывает onCreate при создании Activity. override переопределяет метод родителя, super.onCreate выполняет его базовую часть. Bundle? - возможные сохранённые значения. setContent подключает Compose UI. enableEdgeToEdge разрешает рисовать под системными панелями.

## Экран - описание того, что показывать сейчас

@Composable отмечает функцию UI. Text рисует текст, Image - изображение, Card - карточку. Column размещает детей сверху вниз, Row - рядом, Box может размещать их друг поверх друга. LazyColumn и LazyRow - прокручиваемые списки, которые создают нужные элементы по мере необходимости. Обычная Column сама не прокручивается.

```
Scaffold(
    topBar = {
        TopAppBar(title = { Text(...) })
    },
) { innerPadding ->
    Column(modifier = Modifier.padding(innerPadding)) {
        // Содержимое экрана
    }
}
```

Scaffold - каркас экрана с местами для верхней панели и содержимого. TopAppBar - сама панель. title - заголовок, navigationIcon - кнопка слева, actions - кнопки справа. Scaffold не делает навигацию автоматически.

innerPadding - отступы от панелей и системных краёв, которые Scaffold передаёт содержимому. Их нужно применить самому. Если удалить padding(innerPadding), список может оказаться под верхней панелью. В твоём приложении все три экрана используют Scaffold и TopAppBar.

## Preview и обновление UI

@Preview показывает образец в Android Studio. В preview передают демонстрационные данные и пустые callbacks {}. Нажатия там не доказывают работу настоящей навигации. Ночной Preview задаёт UI_MODE_NIGHT_YES. @OptIn разрешает API, отмеченный экспериментальным в используемой версии Material 3.

Когда наблюдаемое состояние меняется, Compose может повторно выполнить нужные функции UI. Это recomposition, а не перезапуск приложения. Поэтому в теле UI описывают экран, а переходы выполняют в callbacks нажатий.

# 4. Карточка и Modifier: как собран вид

```
OfficeCard
  Card(onClick)
    Row
      Column(weight = 1f)
        Text: название
        Text: город и время
        Row
          RateBlock: покупка
          RateBlock: продажа
      IconButton: сердце
```

Row ставит информацию и сердце рядом. Column внутри него ставит тексты друг под другом. weight(1f) отдаёт Column оставшуюся ширину после места для сердца. Поэтому длинное название не выталкивает кнопку. 1f - Float-коэффициент, не один пиксель.

## Modifier - цепочка настроек элемента

| Настройка | Что делает |
|---|---|
| fillMaxWidth() | Занять доступную ширину родителя. |
| fillMaxSize() | Занять доступную ширину и высоту. |
| padding(...) | Добавить отступ вокруг содержимого. |
| size(...) / height(...) | Задать размер / высоту. |
| heightIn(min = ...) | Задать минимум, допускающий большую высоту. |
| clip(shape) | Обрезать рисунок по форме. |
| weight(1f) | Получить долю оставшегося места в Row/Column. |

dp учитывает плотность дисплея: 48 dp не всегда 48 физических пикселей. Arrangement.spacedBy - промежуток между детьми; Alignment.CenterVertically - их вертикальное выравнивание в Row. contentPadding списка - отступ внутри его прокручиваемого содержимого. Порядок Modifier влияет на измерение и рисунок.

## Три собственных компонента

OfficeCard получает office, isFavorite и два обработчика. Название ограничено maxLines = 1 и TextOverflow.Ellipsis: при нехватке места видно многоточие. Если isFavorite, сердце заполнено; иначе контурное. Карточка используется в List и Favorites.

RateBlock получает label и rate. Surface задаёт его фон и форму, Column размещает подпись и число. Используется в карточке и Details. EmptyState получает icon, title и message; его Column центрирует сообщение. Используется для пустого избранного, фильтра и неизвестного id.

> **Входной modifier передают в корневой элемент компонента. Так экран задаёт его ширину и положение, а компонент - внутренний вид. contentDescription объясняет иконку экранному чтению; у декоративной иконки EmptyState оно null, потому что смысл уже написан рядом.**

# 5. Избранное: состояние и обработчики

```
var favoriteIds by remember { mutableStateOf(setOf<Int>()) }

fun toggleFavorite(officeId: Int) {
    favoriteIds = if (officeId in favoriteIds) {
        favoriteIds - officeId
    } else {
        favoriteIds + officeId
    }
}
```

Set хранит уникальные id. Сначала {}. Нажали сердце у id 3: {3}. Нажали его снова: {}. + и - здесь возвращают новый набор с добавленным или удалённым id. Его присваивание обновляет состояние.

| Часть строки | Её задача |
|---|---|
| mutableStateOf | Создать наблюдаемое значение: Compose замечает запись нового value. |
| remember | Удерживать этот объект между пересчётами UI в той же позиции. |
| by | Читать и записывать state.value через короткое имя favoriteIds. Для этого импортированы getValue/setValue. |

## Проследи одно нажатие

```
Сердце карточки
  -> onFavoriteClick() карточки
  -> { onFavoriteClick(office.id) } экрана
  -> toggleFavorite(id) в NavGraph
  -> новый Set
  -> новый isFavorite и новый список Favorites
```

Callback - переданная функция реакции на событие. Карточка не знает, как хранить избранное: она вызывает переданное действие. Состояние живёт в RateRadarNavGraph, общем родителе трёх экранов. Поэтому одна запись согласованно меняет сердце в списке, кнопку Details и содержимое Favorites. Это называют state hoisting: состояние подняли к общему родителю.

## Почему оно не сохраняется навсегда

remember переживает recomposition, но не обычное пересоздание Activity, например при изменении конфигурации. В рамках живого графа переходы между экранами сохраняют избранное. После полного перезапуска оно пусто: это проверено на эмуляторе.

rememberSaveable дополнительно использует сохранённое состояние Android. В твоём коде так хранится выбранный город. Это тоже не обещание постоянного хранения на любом новом запуске. Базы данных и сохранения избранного на диск сейчас нет.

# 6. Данные, фильтр и список Favorites

```
data class ExchangeOffice(
    val id: Int, val name: String, val city: String,
    val address: String, val workingHours: String,
    val usdBuy: Double, val usdSell: Double, val updatedAt: String,
)
```

data class описывает запись и автоматически даёт сравнение данных, toString и copy. Каждый ExchangeOffice(...) в SampleData создаёт один объект; аргументы идут в порядке полей. object SampleData - единый объект без необходимости писать SampleData(). Его listOf содержит 12 записей. updatedAt - просто текст времени, не реальное обновление.

## Главный экран

```
var selectedCity by rememberSaveable {
    mutableStateOf(SampleData.ALL_CITIES)
}
val offices = if (selectedCity == SampleData.ALL_CITIES) {
    SampleData.offices
} else {
    SampleData.offices.filter { it.city == selectedCity }
}
```

Если город All, показываем исходный список. Иначе filter возвращает только совпавшие записи; исходные данные не удаляются. if здесь возвращает значение выбранной ветки. it - текущий обменник, который проверяет filter.

LazyRow перебирает города. У FilterChip selected = city == selectedCity задаёт активный вид, onClick = { selectedCity = city } меняет состояние. Для Astana останутся id 6-9. Для Karaganda список пуст: показывается EmptyState.

```
items(offices, key = { it.id }) { office ->
    OfficeCard(
        office = office,
        isFavorite = office.id in favoriteIds,
        onClick = { onOfficeClick(office.id) },
        onFavoriteClick = { onFavoriteClick(office.id) },
        modifier = Modifier.fillMaxWidth(),
    )
}
```

items добавляет карточки в LazyColumn. key связывает элемент с постоянным id, а office - объект для текущей карточки. При нажатии передаётся его id. Весь этот блок находится в ветке непустого списка.

## Экран Favorites

```
val favorites = SampleData.offices.filter { it.id in favoriteIds }
```

Он использует тот же OfficeCard. Если набор пуст - EmptyState. Иначе LazyColumn. isFavorite = true у его карточек, потому что записи уже отобраны как избранные. Удалишь последнее сердце - список автоматически станет пустым, и появится сообщение.

# 7. Навигация и экран Details

```
const val DETAILS = "details/{officeId}"
fun details(officeId: Int) = "details/$officeId"

onOfficeClick = { id -> navController.navigate(Routes.details(id)) }
```

Маршрут - адрес экрана внутри приложения. DETAILS задаёт шаблон, details(3) создаёт конкретный адрес «details/3». Карточка вызывает callback, который делает navigate. NavController управляет переходами. NavHost отображает текущий экран. composable(route) регистрирует экран в графе; это не аннотация @Composable.

```
composable(
    route = Routes.DETAILS,
    arguments = listOf(navArgument("officeId") { type = NavType.IntType }),
) { backStackEntry ->
    val officeId = backStackEntry.arguments?.getInt("officeId") ?: 0
    OfficeDetailsScreen(officeId = officeId, ...)
}
```

NavType.IntType объявляет целый аргумент. backStackEntry - запись этого посещения в истории. arguments - контейнер аргументов. ?. безопасно обращается к нему; ?: 0 даёт запасное значение при null. Имя officeId должно совпасть в шаблоне, объявлении аргумента и getInt.

```
val office = SampleData.offices.find { it.id == officeId }
```

Details ищет объект по id, а не по позиции в списке. find возвращает объект или null. При null показываем Office not found; в else Kotlin уже знает, что объект существует, и разрешает office.name. Это уточнение типа называют smart cast.

Внутри Details: Column с verticalScroll(rememberScrollState()), затем Image, название, Card с городом/адресом/часами, Row с двумя RateBlock и кнопка. painterResource загружает рисунок из drawable. Crop заполняет область с возможной обрезкой. Два weight(1f) делят ширину курсов поровну. По isFavorite выбирается Button Add или OutlinedButton Remove.

## Back возвращает к предыдущему посещению

```
[offices] -> [offices, favorites] -> [offices, favorites, details/3]
popBackStack() -> [offices, favorites]
```

Стек закрывает последнее посещение первым. Поэтому из Details, открытого через Favorites, вернёшься в Favorites. Кнопка Back вызывает callback, а NavGraph делает popBackStack. navigate("offices") создал бы новое посещение вместо возврата.

# 8. Цвета, ресурсы и сборка

## Тема работает через роли

```
val TealPrimary = Color(0xFF006A60)
primary = TealPrimary
color = MaterialTheme.colorScheme.primary
```

Это три фрагмента из разных мест: конкретное значение в Color.kt, привязка роли в Theme.kt и её чтение в UI. 0x - шестнадцатеричное число; FF - непрозрачность, остальные пары - RGB. primary - акцент, surface - поверхность карточки, onSurface - цвет текста на ней. onSurface не является обработчиком нажатия.

RateRadarTheme выбирает LightColors или DarkColors по isSystemInDarkTheme и передаёт их MaterialTheme. Цвета UI берутся из colorScheme, стили текста - из typography. Type.kt оставляет стандартные стили Material. Dimens.kt задаёт Spacing 4/8/16/24 dp и Sizes: кнопка 48, иконка 64, картинка 180 dp.

## Откуда берутся надписи и картинки

```
stringResource(R.string.city_and_time, office.city, office.updatedAt)
painterResource(R.drawable.exchange_placeholder)
```

R - сгенерированные id ресурсов, не сами строки. stringResource читает strings.xml: %1$s подставляет первый текстовый аргумент, %2$s - второй. %1$.2f форматирует первый дробный аргумент двумя знаками. painterResource загружает векторный XML-рисунок. В его pathData записана геометрия, а не Kotlin-логика.

| Служебный файл | Что надо понимать |
|---|---|
| AndroidManifest.xml | Объявляет MainActivity. MAIN + LAUNCHER выбирают вход по иконке; exported разрешает launcher запуск. |
| XML themes / values-night | Системная тема окна и её ночной вариант; отдельно от Compose Theme.kt. |
| settings.gradle.kts | Подключает модуль app и репозитории зависимостей. |
| Корневой build.gradle.kts | Версии Android/Kotlin/Compose compiler plugins. |
| app/build.gradle.kts | SDK и библиотеки: Compose, Material 3, Navigation. |
| gradlew + wrapper | Запускают закреплённую версию Gradle. assembleDebug собирает APK. |
| .gitignore / build | Первый исключает локальные файлы из Git, второй хранит результаты сборки. |

minSdk 24 - минимальный Android для установки; compileSdk 36 - API для компиляции; targetSdk 36 - заявленный уровень поведения системы. JVM target 17 относится к bytecode, а не к Android 17. src/test содержит шаблонный JUnit-тест 2+2; он не проверяет UI. sis2 лежит вне Android source set.

# 9. Подготовка: четыре правки и ответы

Сделай эти четыре правки сам, по одной. Запускай приложение и объясняй результат. После упражнения возвращай изменение через Local History или точечную правку; не откатывай чужие изменения целиком.

## 1. Увеличь промежуток между карточками

ExchangeOfficeListScreen -> LazyColumn -> verticalArrangement. Замени Arrangement.spacedBy(Spacing.medium) на Spacing.large. Промежуток станет 16 вместо 8 dp. Менять medium в Dimens.kt для одной области не нужно: это общая настройка.

## 2. Поменяй название верхней панели

В strings.xml измени содержимое title_offices. Text уже использует этот ресурс. Не меняй название функции или имя файла.

## 3. Переставь Buy и Sell

В Row перенеси два вызова RateBlock целиком. Подпись должна остаться со своим курсом: buy с usdBuy, sell с usdSell. Замена только подписей сделает неверный интерфейс.

## 4. Перемести название относительно картинки

В OfficeDetailsScreen внутри Column перенеси весь Text с office.name перед Image или после неё. Порядок детей Column задаёт вертикальный порядок. Текст уже находится под картинкой в исходном коде.

## Ответы, которые нужно уметь объяснить

| Вопрос | Короткий ответ |
|---|---|
| Что такое Scaffold? | Каркас экрана: верхняя панель и область содержимого; innerPadding применяю сам. |
| Как id доходит до Details? | Карточка -> callback -> details/id -> Int-аргумент -> find по id. |
| Где избранное? | Общий Set<Int> в NavGraph; экраны получают данные и callbacks. |
| Зачем remember? | Удерживает состояние между recomposition; постоянного хранения не даёт. |
| Зачем mutableStateOf? | Позволяет Compose заметить запись нового значения. |
| Где цвета? | Значения Color.kt -> роли Theme.kt -> чтение MaterialTheme в UI. |
| Почему Row + Column? | Информация и сердце рядом; строки информации внутри вертикально. |
| Зачем Ellipsis? | Длинное имя не ломает карточку; показываем многоточие. |

> **Пробная защита: покажи три экрана и пустой фильтр; сделай одну из правок; объясни путь id и избранное. Готовность проверяется тем, можешь ли ты найти место изменения и предсказать результат, а не выучить определения дословно.**

# 10. Проверка SIS3: факты и пробелы

Проверены текущие исходники, SIS3.pdf, Git, public GitHub и приложение на отдельном эмуляторе. Код приложения не исправлялся. Сборка, unit test и Lint прошли; Lint дал 0 ошибок и 10 предупреждений. Переходы, кнопки, фильтр, избранное и обе темы проверены в работе.

| Требование | Результат |
|---|---|
| Экраны и layout | Три экрана; Scaffold/TopAppBar; рабочий Back; LazyColumn с 12 записями и LazyRow городов. |
| Состояния контента | Ellipsis в карточках; пустое избранное и Karaganda; ветка неизвестного id; Image из ресурсов. |
| Стиль | Свои light/dark схемы; цветовые роли и Typography; общий Spacing; 48 dp зоны нажатия. |
| Структура | Три собственных компонента, каждый используется повторно; previews light/dark; data class + list. |
| Навигация и state | Navigation Compose, id аргументом и поиск по нему; remember + mutableStateOf. |
| README / GitHub | Public repo, screenshots всех экранов в обеих темах, сравнение с эскизами. |
| Коммиты по времени | Не выполнено: семь SIS3-коммитов сделаны 6 октября с 15:38 до 16:14 по Алматы. |
| Эскизы до кода | До новых экранов SIS3 - да. Но WelcomeScreen уже был в SIS1. PNG есть; требуемый бумажный/Figma-процесс не подтверждён. |
| AI_USAGE / Teams | Разделы AI_USAGE есть, но утверждения о ручной работе должны быть фактическими. Отправку в Teams из проекта проверить нельзя. |

## Небольшие вопросы к точному соответствию

В Details длинное имя переносится полностью: maxLines/overflow там нет. Если преподаватель требует обрезку всех заголовков, добавь их. Описание картинки говорит Photo, хотя это иллюстрация. На эскизе пустого Favorites есть Box, в runtime его заменяет Column компонента; это отличие стоит объяснить в README.

Отсутствие сети, конвертера и базы не нарушает само по себе SIS3: они запланированы в SIS1, а SIS3 оценивает экраны. Дополнительная Karaganda и обработка неизвестного id помогают показать нужные состояния. Запрета на любые дополнительные строки в задании нет.

> **Основной установленный пробел - распределение коммитов по времени. Его не исправить задним числом честным редактированием кода. Остальная оценка зависит и от защиты: нужно самостоятельно объяснять и менять приложение.**

Основа: твои файлы и SIS1/SIS3. Для справки: developer.android.com (Compose state, Scaffold, modifiers, navigation) и kotlinlang.org/docs/basic-syntax.html. Изучай API закреплённых версий проекта; обновление библиотек для подготовки не нужно.